CSharPN

A Code-First Framework for Coloured Petri Net Modelling with C# as Inscription Language

Simon Tjell

Independent Researcher · simon@simplesystemer.dk

PNSE'26 · Petri Nets and Software Engineering · June 2026

The context

A personal note on where this work comes from.

The context

A personal note on where this work comes from.

- Fell for **Coloured Petri Nets** during my MSc and PhD in the CPN group at **University of Aarhus**

The context

A personal note on where this work comes from.

- Fell for **Coloured Petri Nets** during my MSc and PhD in the CPN group at **University of Aarhus**
- Left academia for **industry — 17 years ago**

The context

A personal note on where this work comes from.

- Fell for **Coloured Petri Nets** during my MSc and PhD in the CPN group at **University of Aarhus**
- Left academia for **industry — 17 years ago**
- Systems keep growing more complex: microservices on Kubernetes, with authentication, databases, caches

The context

A personal note on where this work comes from.

- Fell for **Coloured Petri Nets** during my MSc and PhD in the CPN group at **University of Aarhus**
- Left academia for **industry — 17 years ago**
- Systems keep growing more complex: microservices on Kubernetes, with authentication, databases, caches
- The need for **modelling** is very much present — but not always recognised by practitioners

The context

A personal note on where this work comes from.

- Fell for **Coloured Petri Nets** during my MSc and PhD in the CPN group at **University of Aarhus**
- Left academia for **industry — 17 years ago**
- Systems keep growing more complex: microservices on Kubernetes, with authentication, databases, caches
- The need for **modelling** is very much present — but not always recognised by practitioners
- Never forgot the old romance: I kept looking for problems that fit the tool (…), and kept reaching for CPNs

The context

A personal note on where this work comes from.

- Fell for **Coloured Petri Nets** during my MSc and PhD in the CPN group at **University of Aarhus**
- Left academia for **industry — 17 years ago**
- Systems keep growing more complex: microservices on Kubernetes, with authentication, databases, caches
- The need for **modelling** is very much present — but not always recognised by practitioners
- Never forgot the old romance: I kept looking for problems that fit the tool (…), and kept reaching for CPNs
- 🔨 ⊢ ∀p ∈ 🌍 : †(p)

The problem

A persistent **gap** between modelling and implementation:

The problem

A persistent **gap** between modelling and implementation:

- The **modelling tool** (e.g., CPN Tools) sits apart from the **development tool** — the IDE (e.g., Visual Studio Code)

The problem

A persistent **gap** between modelling and implementation:

- The **modelling tool** (e.g., CPN Tools) sits apart from the **development tool** — the IDE (e.g., Visual Studio Code)
- The **modelling language** (e.g. Standard ML) is not the **implementation language** (e.g., C#)

The problem

A persistent **gap** between modelling and implementation:

- The **modelling tool** (e.g., CPN Tools) sits apart from the **development tool** — the IDE (e.g., Visual Studio Code)
- The **modelling language** (e.g. Standard ML) is not the **implementation language** (e.g., C#)
- Model and implementation live in **two different worlds**, and drift apart over time

The problem

A persistent **gap** between modelling and implementation:

- The **modelling tool** (e.g., CPN Tools) sits apart from the **development tool** — the IDE (e.g., Visual Studio Code)
- The **modelling language** (e.g. Standard ML) is not the **implementation language** (e.g., C#)
- Model and implementation live in **two different worlds**, and drift apart over time
- Code generation from model has often been attempted - but has it's own problems

The problem

A persistent **gap** between modelling and implementation:

- The **modelling tool** (e.g., CPN Tools) sits apart from the **development tool** — the IDE (e.g., Visual Studio Code)
- The **modelling language** (e.g. Standard ML) is not the **implementation language** (e.g., C#)
- Model and implementation live in **two different worlds**, and drift apart over time
- Code generation from model has often been attempted - but has it's own problems

Underneath it, the older gap between **academia and industry**.

A possible solution

CSharPN — a framework for working with Coloured Petri Nets in C#, with a VS Code extension for visualising and executing models.

A possible solution

CSharPN — a framework for working with Coloured Petri Nets in C#, with a VS Code extension for visualising and executing models.

- The model is itself a **program in the target language**

A possible solution

CSharPN — a framework for working with Coloured Petri Nets in C#, with a VS Code extension for visualising and executing models.

- The model is itself a **program in the target language**
- Developers stay in their **IDE** (VS Code)

A possible solution

CSharPN — a framework for working with Coloured Petri Nets in C#, with a VS Code extension for visualising and executing models.

- The model is itself a **program in the target language**
- Developers stay in their **IDE** (VS Code)
- Developers work in their **day-to-day language** (C#)

A possible solution

CSharPN — a framework for working with Coloured Petri Nets in C#, with a VS Code extension for visualising and executing models.

- The model is itself a **program in the target language**
- Developers stay in their **IDE** (VS Code)
- Developers work in their **day-to-day language** (C#)
- The **model is the implementation**

A possible solution

CSharPN — a framework for working with Coloured Petri Nets in C#, with a VS Code extension for visualising and executing models.

- The model is itself a **program in the target language**
- Developers stay in their **IDE** (VS Code)
- Developers work in their **day-to-day language** (C#)
- The **model is the implementation**

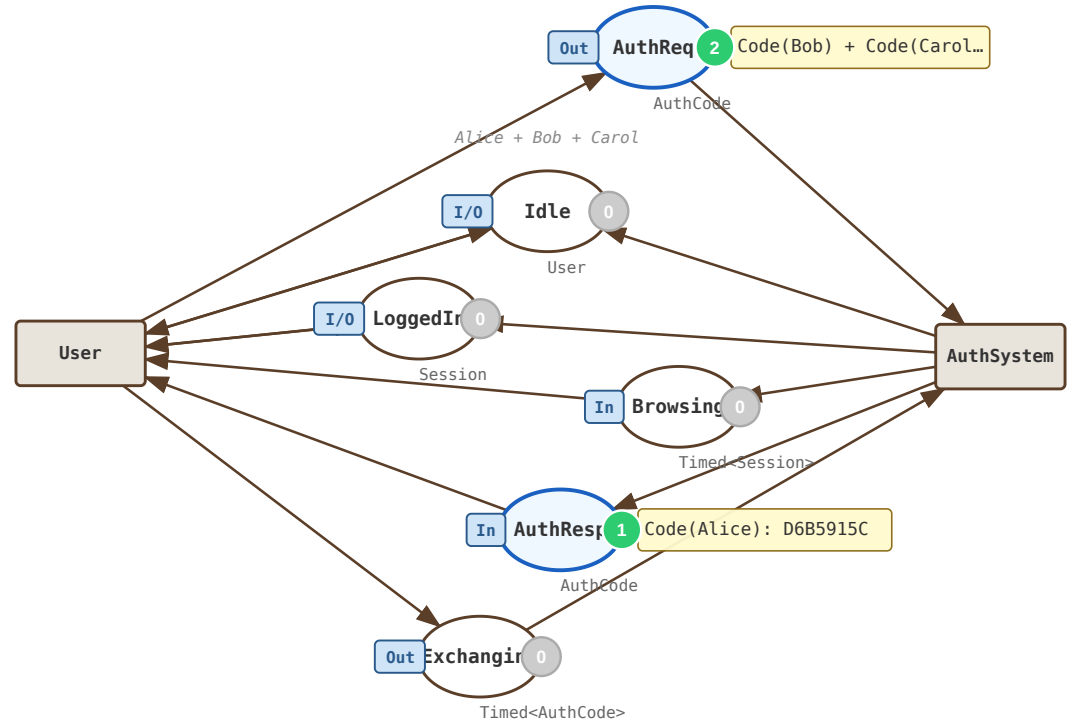
> Simulation-based development — with concurrency, resource sharing, timing, and hierarchy modelled *explicitly*.

Running example:

OAuth2 authorisation code flow

Exercises all four features:

Top page: shared places link the two substitution transitions.



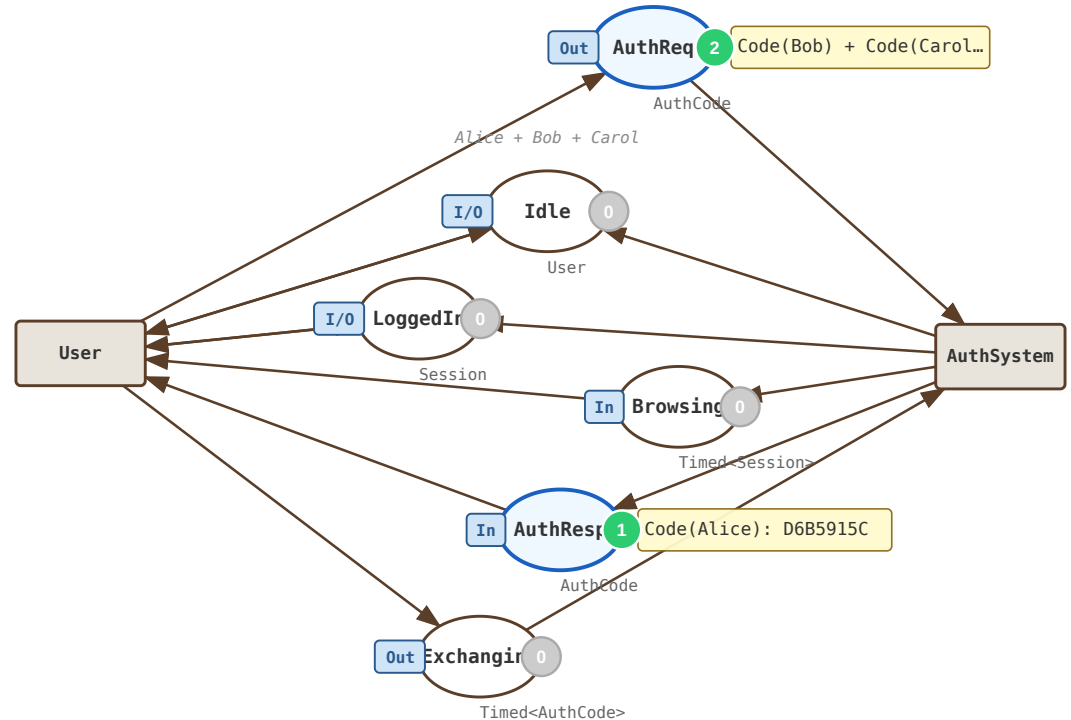
Running example:

OAuth2 authorisation code flow

Exercises all four features:

- Three concurrent users

Top page: shared places link the two substitution transitions.



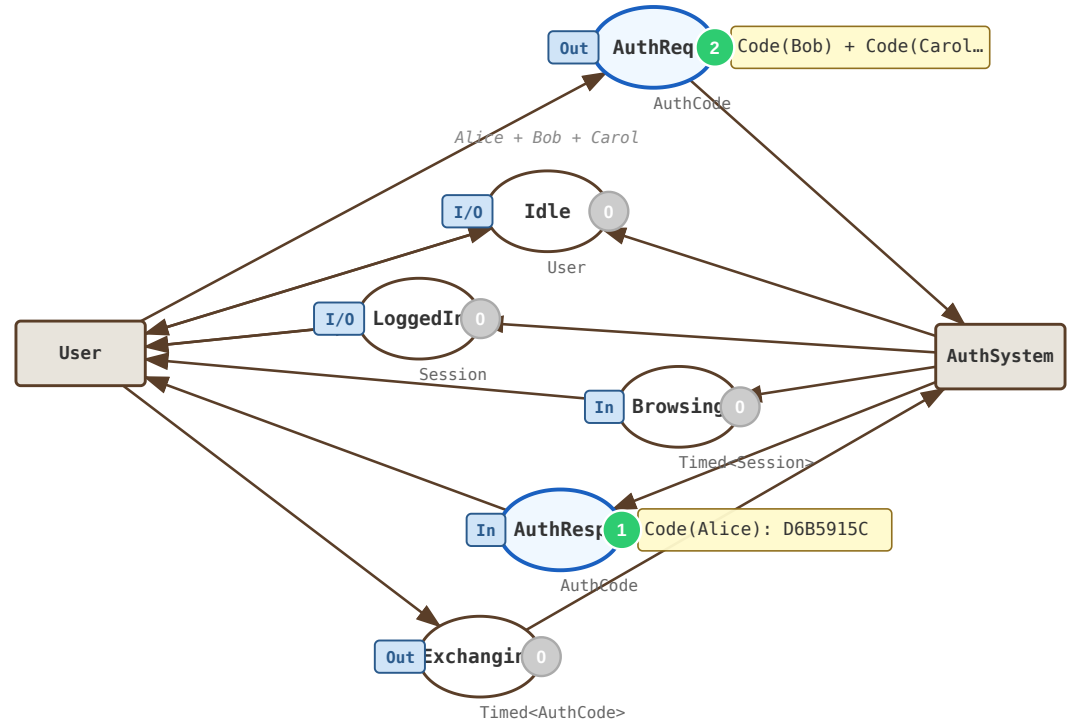
Running example:

OAuth2 authorisation code flow

Exercises all four features:

- Three concurrent users
- Shared places as channels

Top page: shared places link the two substitution transitions.



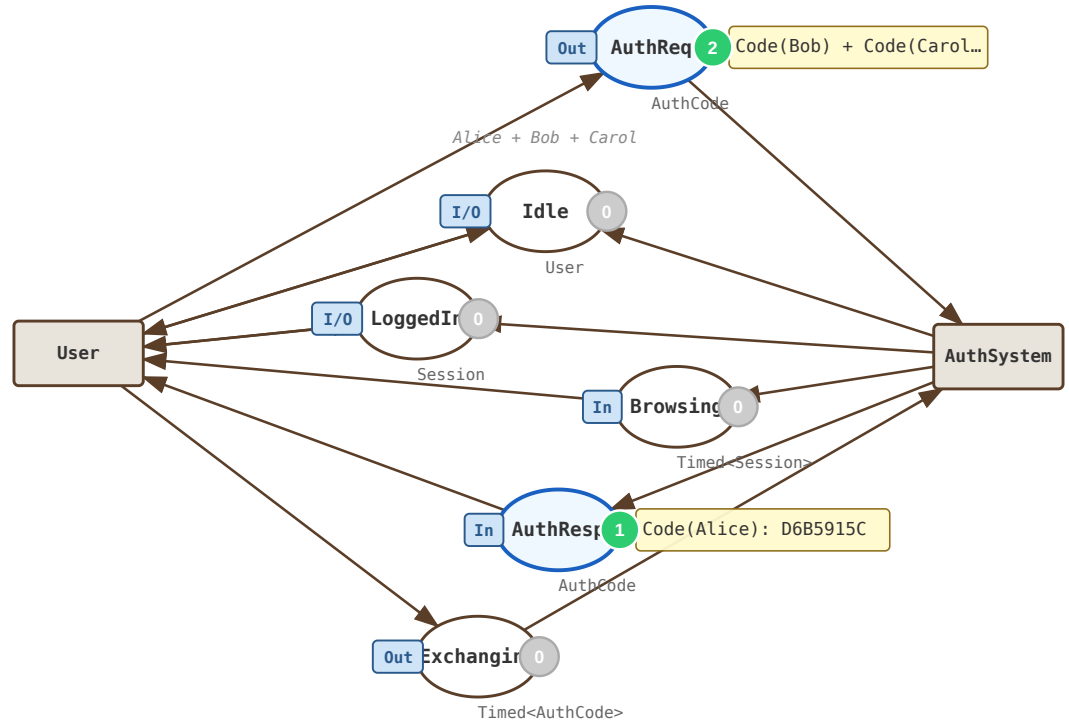
Running example:

OAuth2 authorisation code flow

Exercises all four features:

- Three concurrent users
- Shared places as channels
- Timed auth, exchange, expiry

Top page: shared places link the two substitution transitions.



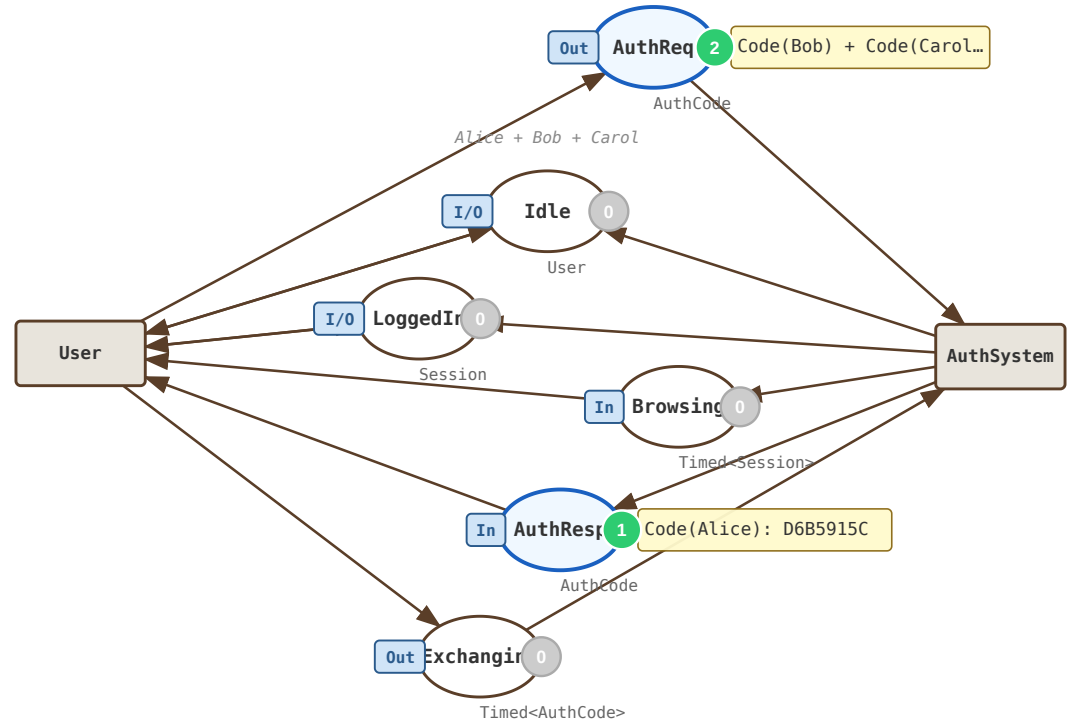
Running example:

OAuth2 authorisation code flow

Exercises all four features:

- Three concurrent users
- Shared places as channels
- Timed auth, exchange, expiry
- Two sub-pages: User & AuthSystem

Top page: shared places link the two substitution transitions.



Colour sets are just C# types

Any `notnull, IEquatable<T>` type works — primitives, enums, records, classes.

```
public record User(int Id, string Name) {
  public string CsrfState  => $"s{Id}";
  public int    BrowseTime => Id * 5;
}

public record AuthCode(User User, string Code, string State) {
  public bool ValidCsrf(User u) => State == u.CsrfState;
  public static AuthCode Create(User u, int seq) {
    var hash = SHA256.HashData(Encoding.UTF8.GetBytes($"{u.Id}:{seq}"));
    return new(u, Convert.ToHexString(hash)[..8], u.CsrfState);
  }
}

public record Session(User User, int TokenId);
```

Colour sets are just C# types

Any `notnull, IEquatable<T>` type works — primitives, enums, records, classes.

```
public record User(int Id, string Name) {
  public string CsrfState  => $"s{Id}";
  public int    BrowseTime => Id * 5;
}

public record AuthCode(User User, string Code, string State) {
  public bool ValidCsrf(User u) => State == u.CsrfState;
  public static AuthCode Create(User u, int seq) {
    var hash = SHA256.HashData(Encoding.UTF8.GetBytes($"{u.Id}:{seq}"));
    return new(u, Convert.ToHexString(hash)[..8], u.CsrfState);
  }
}

public record Session(User User, int TokenId);
```

- **Records** — immutable, with value equality

Colour sets are just C# types

Any `notnull, IEquatable<T>` type works — primitives, enums, records, classes.

```
public record User(int Id, string Name) {
  public string CsrfState  => $"s{Id}";
  public int    BrowseTime => Id * 5;
}

public record AuthCode(User User, string Code, string State) {
  public bool ValidCsrf(User u) => State == u.CsrfState;
  public static AuthCode Create(User u, int seq) {
    var hash = SHA256.HashData(Encoding.UTF8.GetBytes($"{u.Id}:{seq}"));
    return new(u, Convert.ToHexString(hash)[..8], u.CsrfState);
  }
}

public record Session(User User, int TokenId);
```

- **Records** — immutable, with value equality
- Methods carry **domain logic** (`CsrfState` , `BrowseTime`)

Colour sets are just C# types

Any `notnull, IEquatable<T>` type works — primitives, enums, records, classes.

```
public record User(int Id, string Name) {
  public string CsrfState  => $"s{Id}";
  public int    BrowseTime => Id * 5;
}

public record AuthCode(User User, string Code, string State) {
  public bool ValidCsrf(User u) => State == u.CsrfState;
  public static AuthCode Create(User u, int seq) {
    var hash = SHA256.HashData(Encoding.UTF8.GetBytes($"{u.Id}:{seq}"));
    return new(u, Convert.ToHexString(hash)[..8], u.CsrfState);
  }
}

public record Session(User User, int TokenId);
```

- **Records** — immutable, with value equality
- Methods carry **domain logic** (`CsrfState` , `BrowseTime`)
- The **CSRF check** is part of the colour set

Colour sets are just C# types

Any `notnull, IEquatable<T>` type works — primitives, enums, records, classes.

```
public record User(int Id, string Name) {
  public string CsrfState  => $"s{Id}";
  public int    BrowseTime => Id * 5;
}

public record AuthCode(User User, string Code, string State) {
  public bool ValidCsrf(User u) => State == u.CsrfState;
  public static AuthCode Create(User u, int seq) {
    var hash = SHA256.HashData(Encoding.UTF8.GetBytes($"{u.Id}:{seq}"));
    return new(u, Convert.ToHexString(hash)[..8], u.CsrfState);
  }
}

public record Session(User User, int TokenId);
```

- **Records** — immutable, with value equality
- Methods carry **domain logic** (`CsrfState` , `BrowseTime`)
- The **CSRF check** is part of the colour set
- Real cryptography (`SHA256`) inside the model

Colour sets are just C# types

Any `notnull, IEquatable<T>` type works — primitives, enums, records, classes.

```
public record User(int Id, string Name) {
  public string CsrfState  => $"s{Id}";
  public int    BrowseTime => Id * 5;
}

public record AuthCode(User User, string Code, string State) {
  public bool ValidCsrf(User u) => State == u.CsrfState;
  public static AuthCode Create(User u, int seq) {
    var hash = SHA256.HashData(Encoding.UTF8.GetBytes($"{u.Id}:{seq}"));
    return new(u, Convert.ToHexString(hash)[..8], u.CsrfState);
  }
}

public record Session(User User, int TokenId);
```

- **Records** — immutable, with value equality
- Methods carry **domain logic** (`CsrfState` , `BrowseTime`)
- The **CSRF check** is part of the colour set
- Real cryptography (`SHA256`) inside the model

Places & multisets

```
// inside the model constructor
var authReq = AddPlace<AuthCode>("AuthReq");            // empty initial marking

var idle    = AddPlace("Idle", Multiset.Of(
                new User(1,"Alice"), new User(2,"Bob"), new User(3,"Carol")));

var exchg   = AddTimedPlace<AuthCode>("Exchanging");    // timed place
```

Places & multisets

```
// inside the model constructor
var authReq = AddPlace<AuthCode>("AuthReq");            // empty initial marking

var idle    = AddPlace("Idle", Multiset.Of(
                new User(1,"Alice"), new User(2,"Bob"), new User(3,"Carol")));

var exchg   = AddTimedPlace<AuthCode>("Exchanging");    // timed place
```

- `Place<T>` — the type parameter (`T`) specifies the colour set

Places & multisets

```
// inside the model constructor
var authReq = AddPlace<AuthCode>("AuthReq");            // empty initial marking

var idle    = AddPlace("Idle", Multiset.Of(
                new User(1,"Alice"), new User(2,"Bob"), new User(3,"Carol")));

var exchg   = AddTimedPlace<AuthCode>("Exchanging");    // timed place
```

- `Place<T>` — the type parameter (`T`) specifies the colour set
- `Multiset<T>` — immutable, strongly typed bag

Places & multisets

```
// inside the model constructor
var authReq = AddPlace<AuthCode>("AuthReq");            // empty initial marking

var idle    = AddPlace("Idle", Multiset.Of(
                new User(1,"Alice"), new User(2,"Bob"), new User(3,"Carol")));

var exchg   = AddTimedPlace<AuthCode>("Exchanging");    // timed place
```

- `Place<T>` — the type parameter (`T`) specifies the colour set
- `Multiset<T>` — immutable, strongly typed bag
- `AddTimedPlace<T>` → place of `Timed<T>` tokens

Places & multisets

```
// inside the model constructor
var authReq = AddPlace<AuthCode>("AuthReq");            // empty initial marking

var idle    = AddPlace("Idle", Multiset.Of(
                new User(1,"Alice"), new User(2,"Bob"), new User(3,"Carol")));

var exchg   = AddTimedPlace<AuthCode>("Exchanging");    // timed place
```

- `Place<T>` — the type parameter (`T`) specifies the colour set
- `Multiset<T>` — immutable, strongly typed bag
- `AddTimedPlace<T>` → place of `Timed<T>` tokens

Transitions: a fluent builder

```
var u   = new Var<User>("u");

AddTransition("Login")
  .Input(idle, u).Output(waiting, u)
  .Output(authReq, () ⇒ AuthCode.Create(u.Val), "AuthReq")
  .Build();

AddTransition("ReceiveCode")
  .Input(authResp, c).Input(waiting, u)
  .Guard(() ⇒ c.Val.ValidCsrf(u.Val))
  .TimedOutput(exchg, () ⇒ c.Val, Config.ExchangeTime)
  .Build();
```

Transitions: a fluent builder

```
var u   = new Var<User>("u");

AddTransition("Login")
  .Input(idle, u).Output(waiting, u)
  .Output(authReq, () ⇒ AuthCode.Create(u.Val), "AuthReq")
  .Build();

AddTransition("ReceiveCode")
  .Input(authResp, c).Input(waiting, u)
  .Guard(() ⇒ c.Val.ValidCsrf(u.Val))
  .TimedOutput(exchg, () ⇒ c.Val, Config.ExchangeTime)
  .Build();
```

- Variables are strongly typed

Transitions: a fluent builder

```
var u   = new Var<User>("u");

AddTransition("Login")
  .Input(idle, u).Output(waiting, u)
  .Output(authReq, () ⇒ AuthCode.Create(u.Val), "AuthReq")
  .Build();

AddTransition("ReceiveCode")
  .Input(authResp, c).Input(waiting, u)
  .Guard(() ⇒ c.Val.ValidCsrf(u.Val))
  .TimedOutput(exchg, () ⇒ c.Val, Config.ExchangeTime)
  .Build();
```

- Variables are strongly typed
- Transitions are build fluently

Transitions: a fluent builder

```
var u   = new Var<User>("u");

AddTransition("Login")
  .Input(idle, u).Output(waiting, u)
  .Output(authReq, () ⇒ AuthCode.Create(u.Val), "AuthReq")
  .Build();

AddTransition("ReceiveCode")
  .Input(authResp, c).Input(waiting, u)
  .Guard(() ⇒ c.Val.ValidCsrf(u.Val))
  .TimedOutput(exchg, () ⇒ c.Val, Config.ExchangeTime)
  .Build();
```

- Variables are strongly typed
- Transitions are build fluently
- Input arcs may bind variables (or just a multiset of tokens)

Transitions: a fluent builder

```
var u   = new Var<User>("u");

AddTransition("Login")
  .Input(idle, u).Output(waiting, u)
  .Output(authReq, () ⇒ AuthCode.Create(u.Val), "AuthReq")
  .Build();

AddTransition("ReceiveCode")
  .Input(authResp, c).Input(waiting, u)
  .Guard(() ⇒ c.Val.ValidCsrf(u.Val))
  .TimedOutput(exchg, () ⇒ c.Val, Config.ExchangeTime)
  .Build();
```

- Variables are strongly typed
- Transitions are build fluently
- Input arcs may bind variables (or just a multiset of tokens)
- Output arc expressions are **C# lambdas** (or just a variable)

Transitions: a fluent builder

```
var u   = new Var<User>("u");

AddTransition("Login")
  .Input(idle, u).Output(waiting, u)
  .Output(authReq, () ⇒ AuthCode.Create(u.Val), "AuthReq")
  .Build();

AddTransition("ReceiveCode")
  .Input(authResp, c).Input(waiting, u)
  .Guard(() ⇒ c.Val.ValidCsrf(u.Val))
  .TimedOutput(exchg, () ⇒ c.Val, Config.ExchangeTime)
  .Build();
```

- Variables are strongly typed
- Transitions are build fluently
- Input arcs may bind variables (or just a multiset of tokens)
- Output arc expressions are **C# lambdas** (or just a variable)
- Guards are **C# lambdas**

Transitions: a fluent builder

```
var u   = new Var<User>("u");

AddTransition("Login")
  .Input(idle, u).Output(waiting, u)
  .Output(authReq, () ⇒ AuthCode.Create(u.Val), "AuthReq")
  .Build();

AddTransition("ReceiveCode")
  .Input(authResp, c).Input(waiting, u)
  .Guard(() ⇒ c.Val.ValidCsrf(u.Val))
  .TimedOutput(exchg, () ⇒ c.Val, Config.ExchangeTime)
  .Build();
```

- Variables are strongly typed
- Transitions are build fluently
- Input arcs may bind variables (or just a multiset of tokens)
- Output arc expressions are **C# lambdas** (or just a variable)
- Guards are **C# lambdas**

Timed CPNs

```
AddTransition("GrantToken")
  .TimedInput(exchg, c).Input(nextTokId, tid)
  .Output(nextTokId, () ⇒ tid.Val + 1)
  .Output(loggedIn,  () ⇒ new Session(c.Val.User, tid.Val))
  .TimedOutput(expiry,   () ⇒ new Session(c.Val.User, tid.Val), Config.SessionTTL)
    .TimedOutput(browsing, () ⇒ new Session(c.Val.User, tid.Val),
                               () ⇒ c.Val.User.BrowseTime)
  .Build();

AddTransition("SessionExpire")
  .TimedInput(expiry, se).Input(loggedIn, se2)
  .Guard(() ⇒ se.Val.TokenId == se2.Val.TokenId)
  .Output(idle, () ⇒ se.Val.User)
  .Build();
```

Timed CPNs

```
AddTransition("GrantToken")
  .TimedInput(exchg, c).Input(nextTokId, tid)
  .Output(nextTokId, () ⇒ tid.Val + 1)
  .Output(loggedIn,  () ⇒ new Session(c.Val.User, tid.Val))
  .TimedOutput(expiry,   () ⇒ new Session(c.Val.User, tid.Val), Config.SessionTTL)
    .TimedOutput(browsing, () ⇒ new Session(c.Val.User, tid.Val),
                           () ⇒ c.Val.User.BrowseTime)
  .Build();

AddTransition("SessionExpire")
  .TimedInput(expiry, se).Input(loggedIn, se2)
  .Guard(() ⇒ se.Val.TokenId == se2.Val.TokenId)
  .Output(idle, () ⇒ se.Val.User)
  .Build();
```

- Global integer clock; `Timed<T>` tokens carry timestamps

Timed CPNs

```
AddTransition("GrantToken")
  .TimedInput(exchg, c).Input(nextTokId, tid)
  .Output(nextTokId, () ⇒ tid.Val + 1)
  .Output(loggedIn,  () ⇒ new Session(c.Val.User, tid.Val))
  .TimedOutput(expiry,   () ⇒ new Session(c.Val.User, tid.Val), Config.SessionTTL)
    .TimedOutput(browsing, () ⇒ new Session(c.Val.User, tid.Val),
                           () ⇒ c.Val.User.BrowseTime)
  .Build();

AddTransition("SessionExpire")
  .TimedInput(expiry, se).Input(loggedIn, se2)
  .Guard(() ⇒ se.Val.TokenId == se2.Val.TokenId)
  .Output(idle, () ⇒ se.Val.User)
  .Build();
```

- Global integer clock; `Timed<T>` tokens carry timestamps
- Timed output tokens are ready after a constant or dynamic delay

Timed CPNs

```
AddTransition("GrantToken")
  .TimedInput(exchg, c).Input(nextTokId, tid)
  .Output(nextTokId, () ⇒ tid.Val + 1)
  .Output(loggedIn,  () ⇒ new Session(c.Val.User, tid.Val))
  .TimedOutput(expiry,   () ⇒ new Session(c.Val.User, tid.Val), Config.SessionTTL)
    .TimedOutput(browsing, () ⇒ new Session(c.Val.User, tid.Val),
                           () ⇒ c.Val.User.BrowseTime)
  .Build();

AddTransition("SessionExpire")
  .TimedInput(expiry, se).Input(loggedIn, se2)
  .Guard(() ⇒ se.Val.TokenId == se2.Val.TokenId)
  .Output(idle, () ⇒ se.Val.User)
  .Build();
```

- Global integer clock; `Timed<T>` tokens carry timestamps
- Timed output tokens are ready after a constant or dynamic delay
- `SessionExpire` consumes the expiry timer when it matures — races with `Logout` (in `UserPage`) for the same session

Timed CPNs

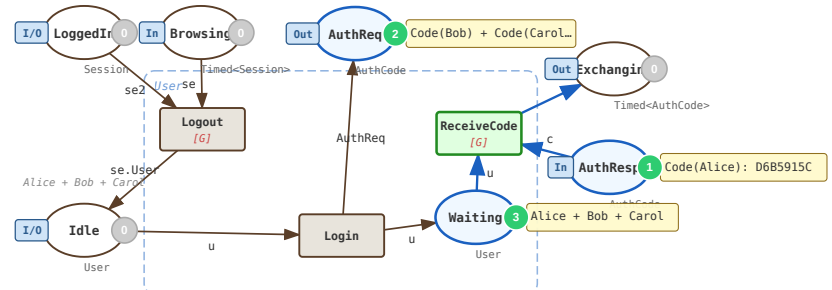
```
AddTransition("GrantToken")
  .TimedInput(exchg, c).Input(nextTokId, tid)
  .Output(nextTokId, () ⇒ tid.Val + 1)
  .Output(loggedIn,  () ⇒ new Session(c.Val.User, tid.Val))
  .TimedOutput(expiry,   () ⇒ new Session(c.Val.User, tid.Val), Config.SessionTTL)
    .TimedOutput(browsing, () ⇒ new Session(c.Val.User, tid.Val),
                               () ⇒ c.Val.User.BrowseTime)
  .Build();

AddTransition("SessionExpire")
  .TimedInput(expiry, se).Input(loggedIn, se2)
  .Guard(() ⇒ se.Val.TokenId == se2.Val.TokenId)
  .Output(idle, () ⇒ se.Val.User)
  .Build();
```

- Global integer clock; `Timed<T>` tokens carry timestamps
- Timed output tokens are ready after a constant or dynamic delay
- `SessionExpire` consumes the expiry timer when it matures — races with `Logout` (in `UserPage`) for the same session

Hierarchical CPNs

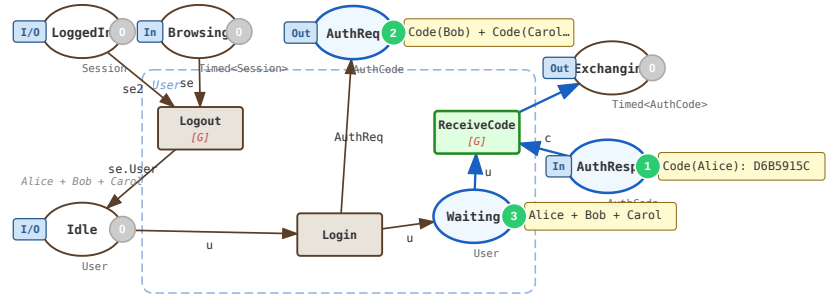
```
class UserPage : CpnPage {
  public UserPage(Place<User> idle,
       Place<AuthCode> authReq, ...)
    : base("User") {
    InOut(idle);
    Out(authReq); In(authResp);
    Out(exchg);
    InOut(loggedIn); In(browsing);
    var waiting = AddPlace<User>("Waiting");
    // Login, ReceiveCode, Logout
  }
}
```




```
AddSubPage(new UserPage(...),       "User");
AddSubPage(new AuthSystemPage(...), "AuthSystem");
```

Hierarchical CPNs

```
class UserPage : CpnPage {
  public UserPage(Place<User> idle,
       Place<AuthCode> authReq, ...)
    : base("User") {
    InOut(idle);
    Out(authReq); In(authResp);
    Out(exchg);
    InOut(loggedIn); In(browsing);
    var waiting = AddPlace<User>("Waiting");
    // Login, ReceiveCode, Logout
  }
}
```

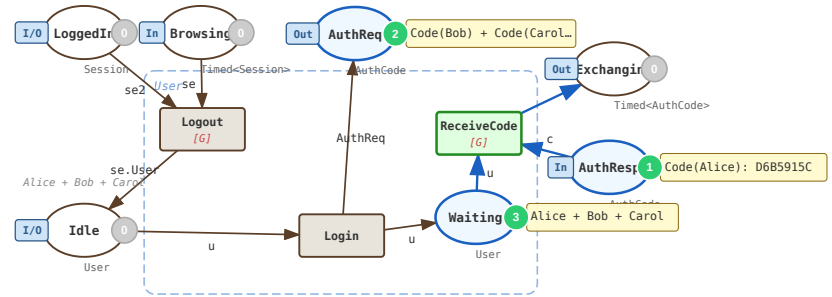


- A sub-page is a `CpnPage` **subclass**

```
AddSubPage(new UserPage(...),       "User");
AddSubPage(new AuthSystemPage(...), "AuthSystem");
```

Hierarchical CPNs

```
class UserPage : CpnPage {
  public UserPage(Place<User> idle,
       Place<AuthCode> authReq, ...)
    : base("User") {
    InOut(idle);
    Out(authReq); In(authResp);
    Out(exchg);
    InOut(loggedIn); In(browsing);
    var waiting = AddPlace<User>("Waiting");
    // Login, ReceiveCode, Logout
  }
}
```

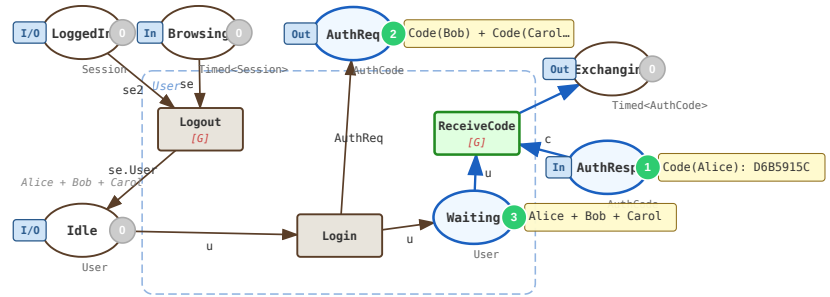


- A sub-page is a `CpnPage` **subclass**
- Ports are declared using `In()` / `Out()` / `InOut()`

```
AddSubPage(new UserPage(...),       "User");
AddSubPage(new AuthSystemPage(...), "AuthSystem");
```

Hierarchical CPNs

```
class UserPage : CpnPage {
  public UserPage(Place<User> idle,
       Place<AuthCode> authReq, ...)
    : base("User") {
    InOut(idle);
    Out(authReq); In(authResp);
    Out(exchg);
    InOut(loggedIn); In(browsing);
    var waiting = AddPlace<User>("Waiting");
    // Login, ReceiveCode, Logout
  }
}
```



- A sub-page is a `CpnPage` **subclass**
- Ports are declared using `In()` / `Out()` / `InOut()`

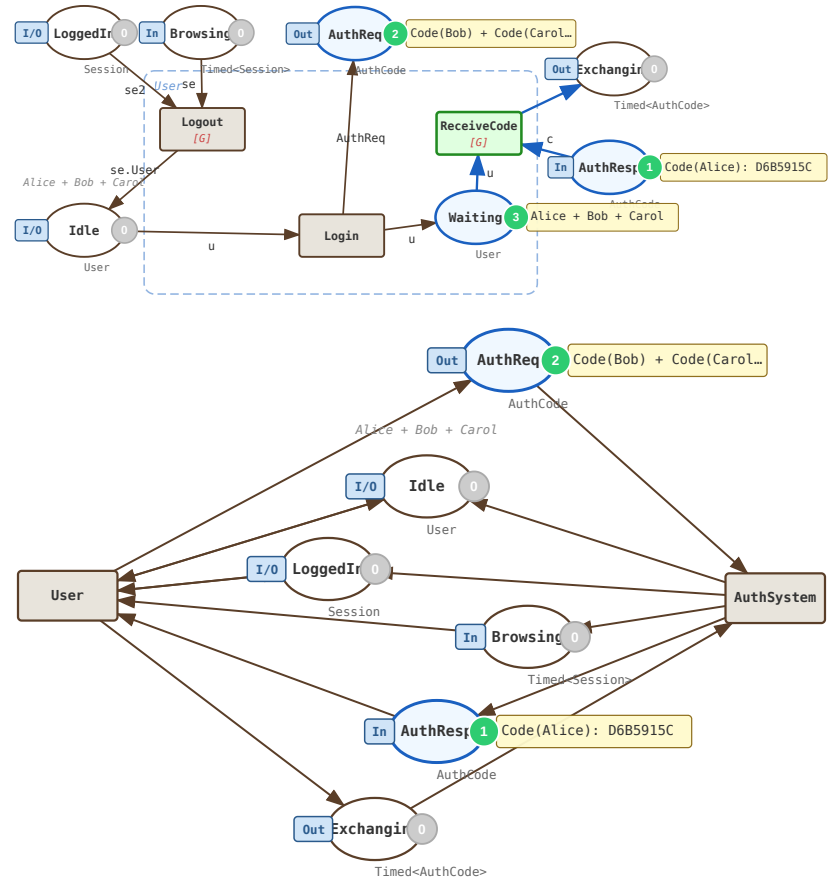
```
AddSubPage(new UserPage(...),       "User");
AddSubPage(new AuthSystemPage(...), "AuthSystem");
```

Hierarchical CPNs

```
class UserPage : CpnPage {
  public UserPage(Place<User> idle,
       Place<AuthCode> authReq, ...)
    : base("User") {
    InOut(idle);
    Out(authReq); In(authResp);
    Out(exchg);
    InOut(loggedIn); In(browsing);
    var waiting = AddPlace<User>("Waiting");
    // Login, ReceiveCode, Logout
  }
}
```

- A sub-page is a `CpnPage` **subclass**
- Ports are declared using `In()` / `Out()` / `InOut()`
- `AddSubPage` wires it into the parent:

```
AddSubPage(new UserPage(...),       "User");
AddSubPage(new AuthSystemPage(...), "AuthSystem");
```

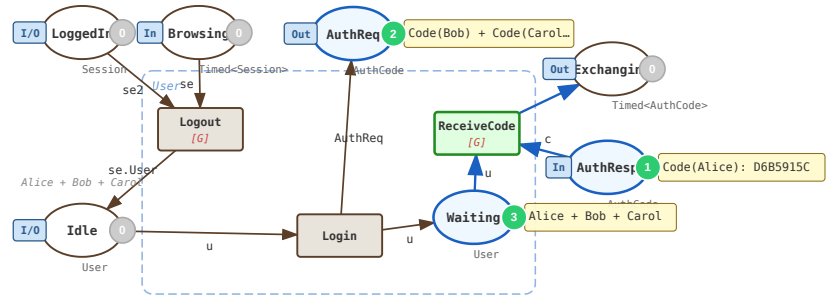


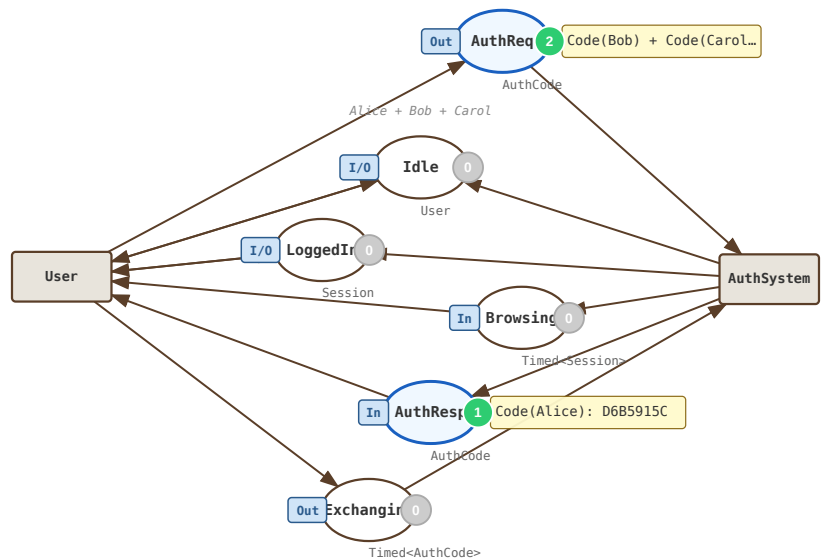
Hierarchical CPNs

```
class UserPage : CpnPage {
  public UserPage(Place<User> idle,
       Place<AuthCode> authReq, ...)
    : base("User") {
    InOut(idle);
    Out(authReq); In(authResp);
    Out(exchg);
    InOut(loggedIn); In(browsing);
    var waiting = AddPlace<User>("Waiting");
    // Login, ReceiveCode, Logout
  }
}
```

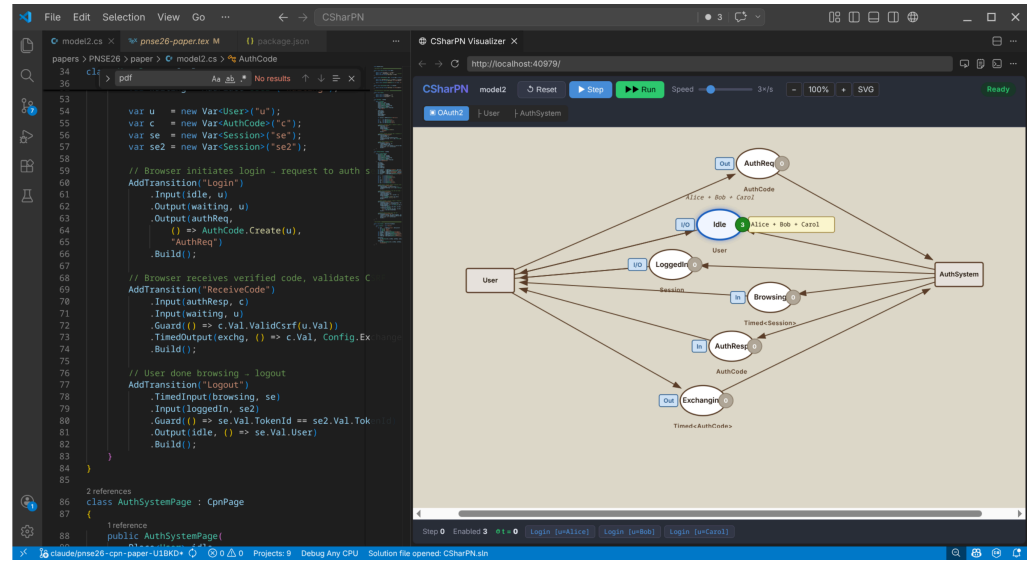
- A sub-page is a `CpnPage` **subclass**
- Ports are declared using `In()` / `Out()` / `InOut()`
- `AddSubPage` wires it into the parent:

```
AddSubPage(new UserPage(...),       "User");
AddSubPage(new AuthSystemPage(...), "AuthSystem");
```



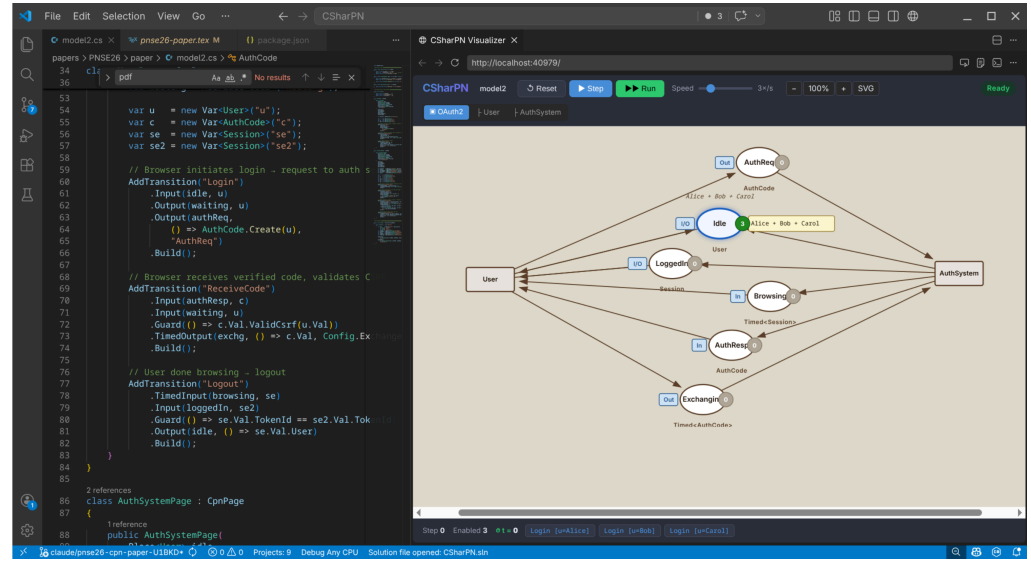


Visualiser & VS Code integration



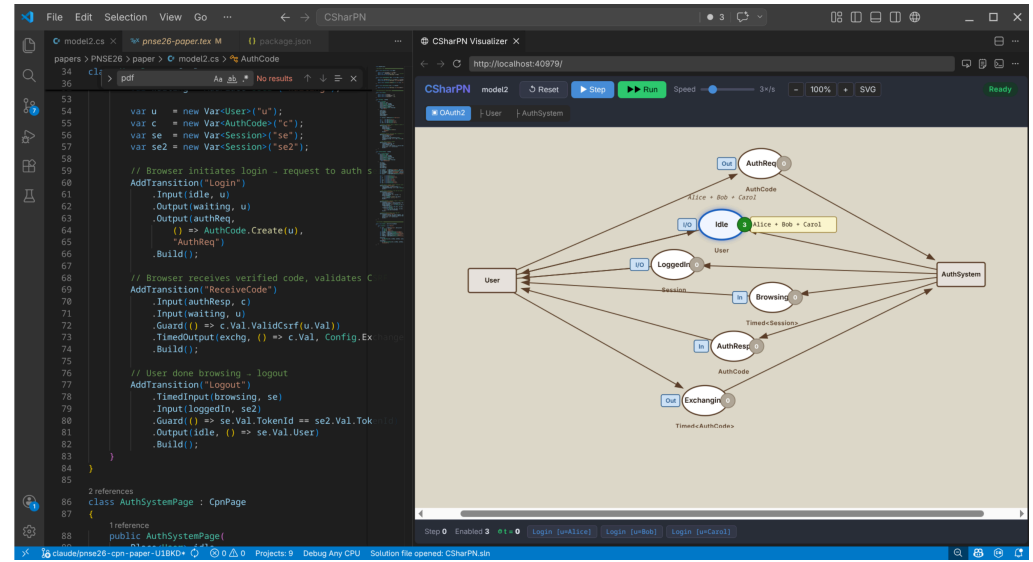
Visualiser & VS Code integration

- Blazor app in VS Code's Simple Browser panel



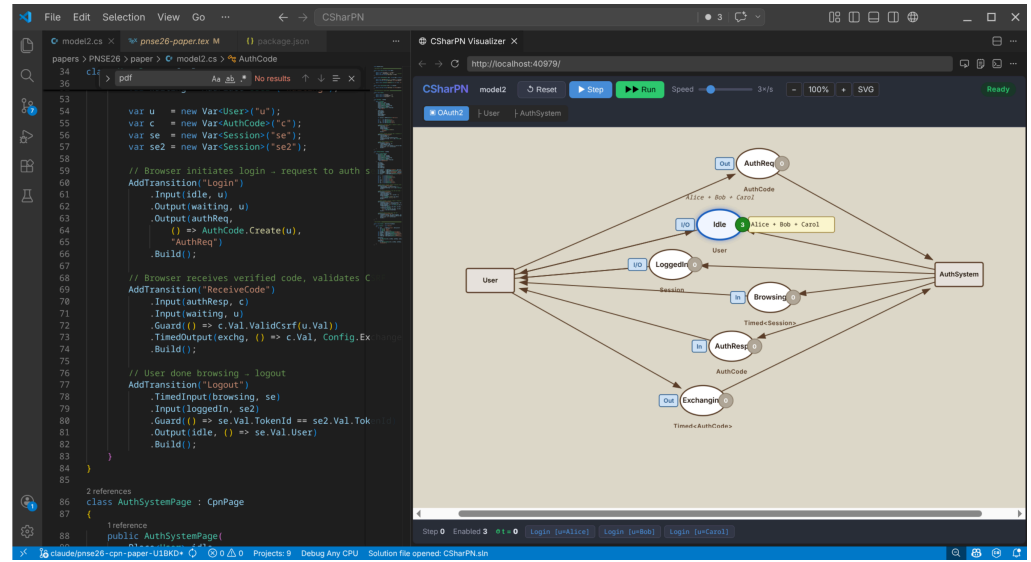
Visualiser & VS Code integration

- Blazor app in VS Code's Simple Browser panel
- Page tree — drill into sub-pages



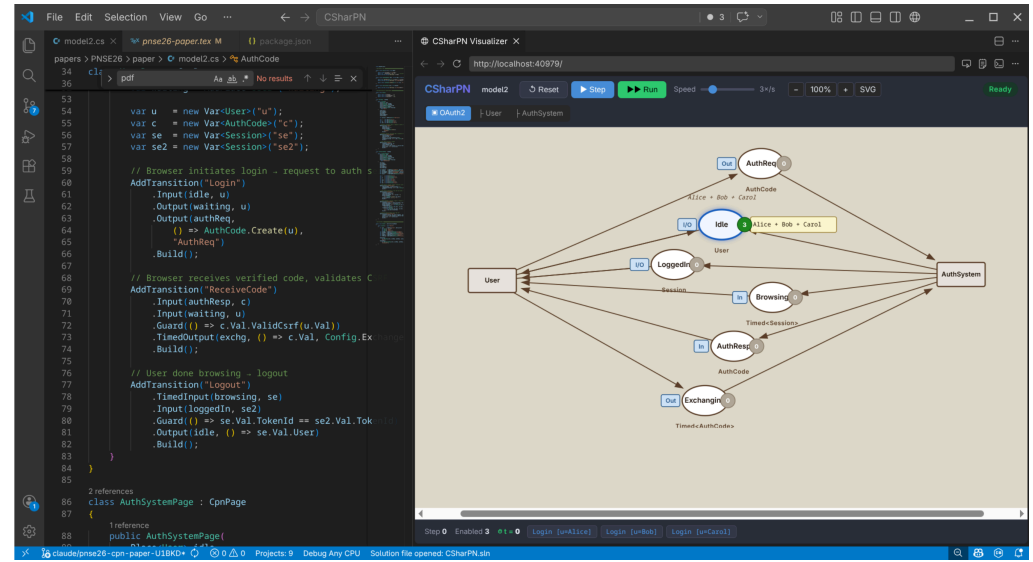
Visualiser & VS Code integration

- Blazor app in VS Code's Simple Browser panel
- Page tree — drill into sub-pages
- Clock, step count, enabled bindings



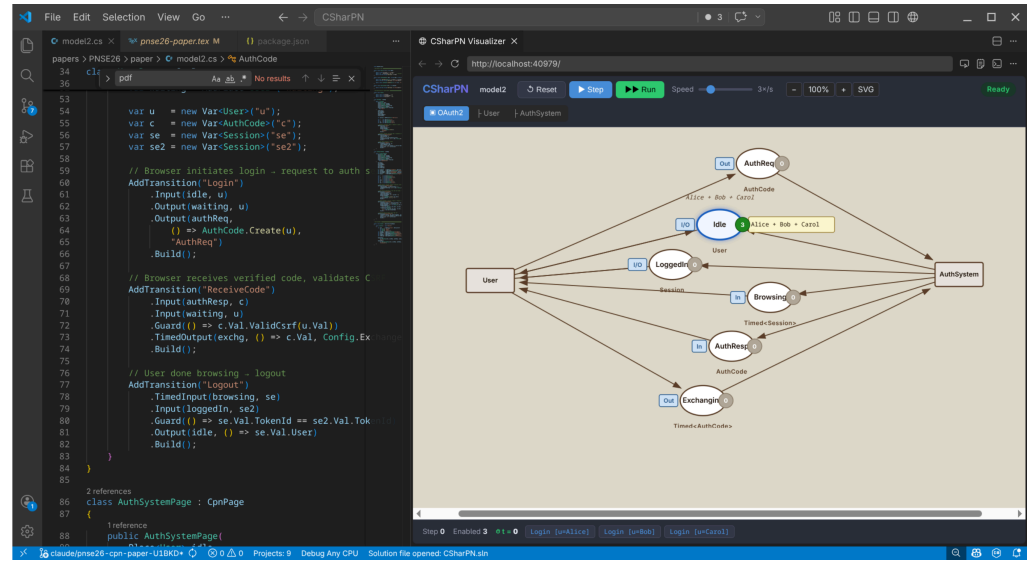
Visualiser & VS Code integration

- Blazor app in VS Code's Simple Browser panel
- Page tree — drill into sub-pages
- Clock, step count, enabled bindings
- Click a binding to fire it



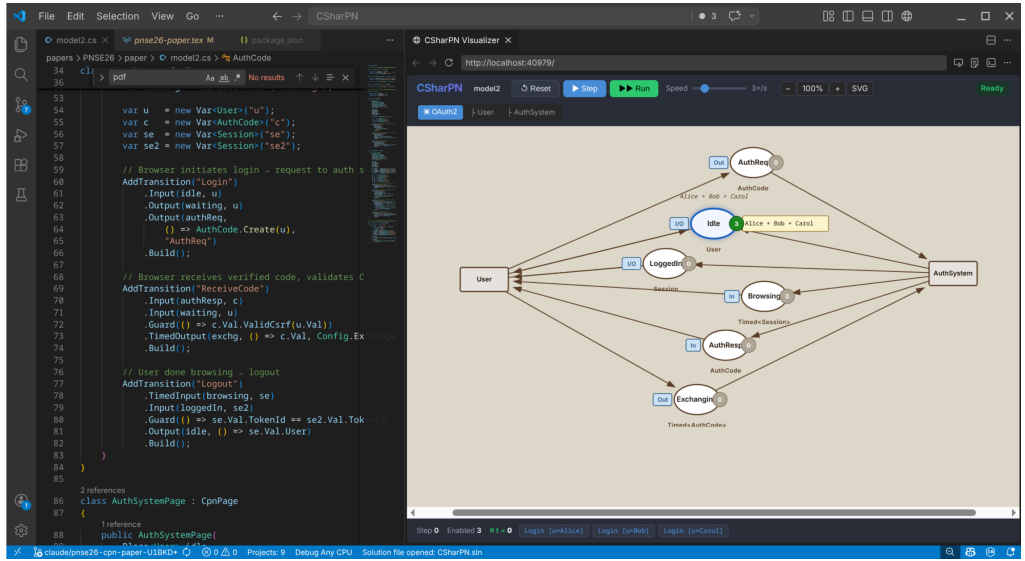
Visualiser & VS Code integration

- Blazor app in VS Code's Simple Browser panel
- Page tree — drill into sub-pages
- Clock, step count, enabled bindings
- Click a binding to fire it
- **Double-click** a place/transition → jump to source line



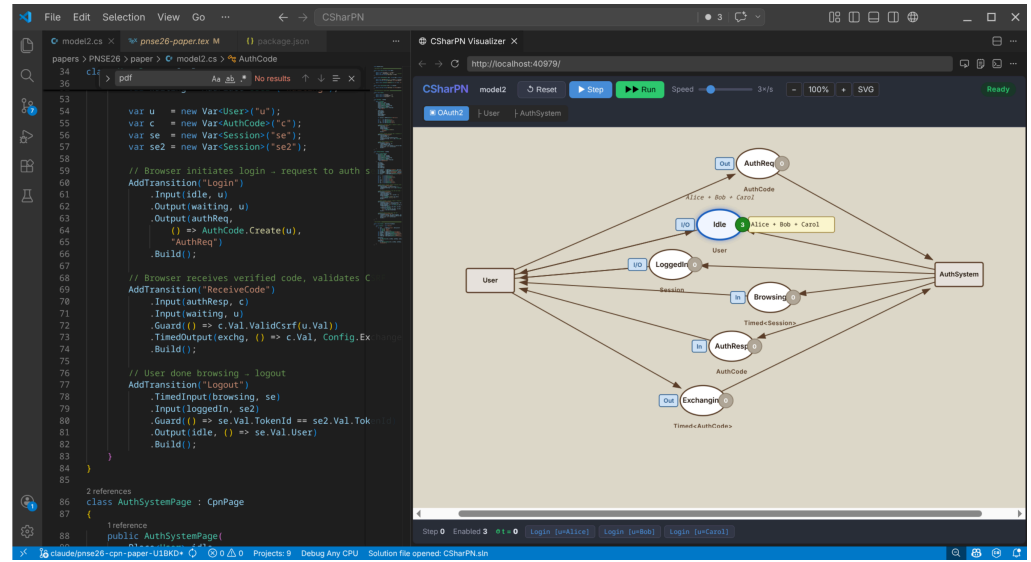
Visualiser & VS Code integration

- Blazor app in VS Code's Simple Browser panel
- Page tree — drill into sub-pages
- Clock, step count, enabled bindings
- Click a binding to fire it
- **Double-click** a place/transition → jump to source line
- Layout persisted in `model.cs.layout`



Visualiser & VS Code integration

- Blazor app in VS Code's Simple Browser panel
- Page tree — drill into sub-pages
- Clock, step count, enabled bindings
- Click a binding to fire it
- **Double-click** a place/transition → jump to source line
- Layout persisted in `model.cs.layout`
- **Hot reload**: save .cs → recompile → reload, marking preserved



Future work

Future work

- Models as deployable .NET components (HTTP-driven transitions/places)

Future work

- Models as deployable .NET components (HTTP-driven transitions/places)
- Specialized places (e.g., database-bound)

Future work

- Models as deployable .NET components (HTTP-driven transitions/places)
- Specialized places (e.g., database-bound)
- CPN-as-test-harness for existing C# libraries

Future work

- Models as deployable .NET components (HTTP-driven transitions/places)
- Specialized places (e.g., database-bound)
- CPN-as-test-harness for existing C# libraries
- State-space analysis on `CpnState` snapshots

Thank you

Questions?

CSharPN — MIT licensed

`github.com/simontjell/CSharPN.public`

Simon Tjell · simon@simplesystemer.dk